

Лекція 1. Вступ та початок роботи з мовою Java

1. Вступ до Java. Типи середовищ та версії. (Java SE, Java EE).
2. Компілятори Java. JVM, JDK, JRE, IDE. Встановлення та запуск JDK
3. Створення та запуск простої програми в блокноті.
4. Введення-виведення. Методи класу System.out: print, println.
5. Побудова оточення та настройка ECLIPSE
6. Процес компіляції Java-програми
7. Робота з байт-кодом
8. Відлагодження програм на Java

Вступ до Java

Java — це мова програмування, що призначення для створення платформонезалежних об'єктно-орієнтованих застосувань з використанням Сі-подібного синтаксису.

Переваги цієї мови:

- Java - це проста мова: Java легко вивчити, а її синтаксис - чіткий і стислий. Він заснований на C ++ . Тому програмістам, які знають C ++ легко перейти на цю мову. Java видала багато складних або рідко використовуваних функцій, наприклад явні вказівники, перевантаження операторів тощо. Java також піклується про управління пам'яттю, а також забезпечує автоматичний збір сміття. Вона автоматично прибирає невикористовувані об'єкти.

- Java - це не залежна від платформи мова: програми, написані мовою Java, після компіляції перетворюються на мову проміжного рівня, яку називають байт-кодом, яка є окремою від платформи Java незалежно від машини, на якій працюють програми. Це робить Java високо портативною, оскільки її байт-коди можуть бути запущені на будь-якій машині інтерпретатором, який називається Java Virtual Machine (JVM), і таким чином Java забезпечує "повторне використання коду".

- Java - об'єктно-орієнтована мова програмування: ООП робить програму, поділяючи її на ряд об'єктів. Об'єкти можуть бути використані як шлях для передачі даних від однієї функції до іншої. Отже, можна легко змінювати дані та функції відповідно до вимог програми.

- Java є надійною мовою: програми Java повинні бути надійними, оскільки вони використовуються як у споживчих, так і в критичних для роботи програмах, починаючи від плеєрів Blu-ray і навігаційних систем.

- Java - це багатопотокова мова: Java може виконувати багато завдань одночасно, визначаючи кілька потоків. Наприклад, програма, яка керує графічним інтерфейсом користувача (GUI) під час очікування введення з мережевого з'єднання, використовує інший потік для виконання та чекання замість використання потоку GUI за замовчуванням для обох завдань. Це дозволяє GUI реагувати в реальному часі.

- Java-програми можуть створювати аплети: аплети - це програми, які працюють у веб-браузерах.
- Java не вимагає жодного препроцесора: для створення програми Java не потрібно включати файли заголовків.

Насправді можна розглядати Java не лише як мову, а як певний стек технологій, що стосуються різних галузей програмних застосувань.

Класифікація технологій Java:

1. Веб-розробка

Веб-розробка включає в себе розробку веб-сайту (колекцій веб-сторінок) або веб-додатку (колекція менших додатків / модулів, що схожа на веб-сайт).

При цьому технології, що відносяться до Java можуть стосуватись як засоби створення клієнту, так і, що більш складніше, серверні технології.

Технології на стороні сервера повинні дозволяти збирати дані з фронтенду, а потім передавати їх на логічний рівень, а після обробки повертати відповідь на фронтенд - це робота бекенда. Це стосується технологій на стороні сервера.

Найбільш популярні Java фронтенд фреймворкі:

- Spring framework

Цей фреймворк пропонує спосіб розробити масштабований веб-додаток за допомогою архітектури MVC (model-view-controller). Близько 60% IT-галузей використовують Spring. Spring також передбачає стандартні заходи безпеки.

- Spring Boot

Spring Boot - це найпоширеніший фреймворк, який зменшує головний біль у виконанні такої конфігурації, як у Spring.

- Grails

Це швидка рамка розробки додатків, побудована Spring. Це забезпечує певні переваги Spring різом зі швидким розвитком середовища, що є важливим для користувача. Крім того, він має видатну підтримку громади.

2. Сервери додатків Java

Сервер - це в основному система, яка обробляє HTTP-запити і повертає відповідь (може бути веб-сторінкою або даними). Нижче наведено декілька найвідоміших локальних серверів, що використовуються з фреймворками Java:

- Tomcat

Tomcat - найпопулярніший сервер додатків Java. Існує стандарт Java для архіву веб-додатків - ака WAR-файл. Вони розміщені на серверах прикладних програм, таких як Tomcat, щоб забезпечити середовище виконання веб-програми. Також додатки Java можуть бути розгорнуті і в хмарі.

3. Додатки для Android

Програми Android - це програми .apk, які працюють на пристроях Android. Сьогодні все більше пристроїв мають ядро Android.

IDE / Інструменти для розробки додатків Android

Нижче наведено кілька інструментів, які надають багатий інтерфейс для розробки додатків для Android.

- Android Studio
- Eclipse

Це одне з найвідоміших IDE (інтегрованих середовищ розробки) для роботи над технологіями на базі Java. Він має широку підтримку плагінів для різних видів діяльності, включаючи розробку Android.

- AVD Manager

AVD Manager - емулятор Android.

- IntelliJ IDEA

Як і Eclipse та Netbeans, це також дуже відомий інструмент розробки Java.

3. Робота з великими даними (BigData)

Оскільки дані зростають з кожним днем, для кожної організації стає необхідним обробляти дані. Будь то тренування статистичної моделі чи чогось іншого, обробляти великі дані непросто, а значить, для цього потрібні певні інструменти.

Деякі з них наведені нижче.

Інструменти для великих даних

Як і Python, Java також пропонує неймовірні інструменти для аналізу великих даних.

- HADOOP

Це інструмент з відкритим кодом, побудований над Java та використовується для зберігання та обробки великих даних. Він сьогодні дуже широко використовується.

- Spark

Spark використовується для передачі величезних даних та розподілу часних даних.

- Kafka

Kafka використовується для встановлення черги великих обсягів інформації.

- Cassandra

Cassandra використовується для запитів та зберігання великих / величезних даних.

- Spring Boot

Spring Boot використовується для надання клієнтам параметрів системи через REST API. Незважаючи на те, що це резервна основа, її можна комбінувати з іншими інструментами для аналізу великих даних.

- Elasticsearch

Elasticsearch використовується для зберігання та запиту масових обсягів інформації.

4. DevOps

Java DevOps — це стек технологій для розробки програмного забезпечення, що забезпечує:

- Постійну інтеграцію

У цьому процесі оновлена версія коду інтегрується через певний проміжок часу (наприклад: - у спринтах та відповідній методології). Тестування та збирання слід робити після кожної версії, а отже, і помилки видаляються на певних етапах, а отже, код залишається чистим і не містить помилок.

- Постійне розгортання

Тож після того, як код буде інтегрований та протестований, його потрібно створити та розгорнути на виробництві ще на стадії розробки.

- Мікросервіси

Замість великих додатків всю програму можна скласти з менших додатків.

Наприклад, коли ми створюємо веб-додаток, то він може бути виготовлений з безлічі вільно з'єднаних, легко самостійно розгорнутих деталей. Наприклад, веб-додаток, що відповідає архітектурі MVC.

5. Комп'ютерні Додатки (Desktop Applications)

Для створення додатків на Java, що використовують графічні інтерфейси можна використовувати наступні пакети Java:

- | | |
|----|--------|
| 1. | Swing |
| 2. | AWT |
| 3. | JavaFX |

6. Також мову Джава можна використовувати в настурних застосуваннях:

- Незалежні програмні пакети (runnable jars)
- Багато популярних APIs та сервісів, такі як AWS/AZURE/GCP мають SDK на Java.
- Kotlin -Java-based мова

Типи середовищ та версії. (Java SE, Java EE)

Спочатку Java (офіційний день народження технології Java - 23 травня 1995 г.) призначалася для програмування побутових електронних пристроїв, таких, як телефони. Потім Java стала застосовуватися для створення активних гіпертекстових сторінок - з'явилися аплеті. Потім виявилось, що на Java можна створювати повноцінні програми. Їх графічні елементи стали оформляти у вигляді компонентів - з'явилися JavaBeans, з якими Java увійшла в світ розподілених систем і проміжного програмного забезпечення, тісно зв'язавшись з технологією CORBA. Залишився один крок до програмування серверів - цей крок був зроблений - з'явилися сервлети і EJB (Enterprise JavaBeans). Сервери повинні взаємодіяти з системами управління базами даних

(СКБД) - з'явилися драйвери JDBC (Java DataBase Connection). Багато СУБД і навіть операційні системи (ОС) включили Java в своє ядро, наприклад, Oracle, Linux. Сьогодні можна сказати, що Java - технологія, яка швидко охоплює все нові і нові області.

Основні підмови та версії мови Java

Наразі версії Java поділяють на ранні Java1 (JDK 1.1) та більш сучасні Java2 – це версії починаючи з другої J2SE 1.2. До п'ятої версії нумерація версій шла J2SE 1.2 - J2SE 1.4, починаючи з п'ятої - J2SE 5.0. Наразі найбільш стабільна сучасна версія – це восьма версія компілятора мови, хоча вже існує J2SE 12.

В мові Java також виділяють певні підмножини, що визначають певним чином технологію та стиль програмування на Java:

Java SE (Standard Edition) - J2SE

Java EE (Enterprise Edition) J2EE

Java ME (Micro Edition), J2ME

J2SE призначена для використання на робочих станціях і персональних комп'ютерах. Насправді, Standard Edition - основа технології Java та прямий розвиток JDK (яке можна також назвати j2sdk).

J2EE містить все для створення складних, високонадійних розподілених серверних застосунків. Умовно можна сказати, що Enterprise Edition - це набір потужних бібліотек (наприклад, Enterprise Java Beans, EJB) і приклади реалізацій платформи (сервера застосунків, Application Server), яка їх підтримує. Робота такої платформи завжди опирається на j2sdk.

J2ME є зменшеною версією Standard Edition для того, щоб задовільняти жорстким апаратним вимогам невеликих пристроїв, таких як карманні комп'ютери та стільникові телефони.

JVM, JDK, JRE, IDE. Встановлення та запуск JDK

Java - це комп'ютерна мова програмування загального призначення, яка є багатопотоковою, об'єктно-орієнтованою тощо.

Програми Java зазвичай компілюються в байт-код, який може працювати на будь-якій віртуальній машині Java (JVM) незалежно від архітектури комп'ютера.

Нижче наведено налаштування середовища для Linux та Windows. Усі три JVM, JRE та JDK залежать від платформи, оскільки конфігурація кожної операційної системи різна. Але, Java не залежить від платформи.

Перед тим, як створити навколишнє середовище, необхідно зрозуміти наступні поняття:

1. JDK (Java Development Kit, середовище розробки): JDK призначений для розробників програмного забезпечення та включає в себе інструменти розробки, такі як компілятор Java, Javadoc, Jar та відладчик.
2. JRE (Java Runtime Environment, середовище виконання Java): JRE містить частини бібліотек Java, необхідні для запуску програм Java, і призначені для кінцевих користувачів. JRE можна розглядати як підмножину JDK.
3. JVM (Java Virtual Machine, віртуальна машина Java) - це абстрактна машина. Це специфікація, яка забезпечує середовище виконання, в якому може виконуватися байт-код Java. JVM доступні для багатьох апаратних та програмних платформ.

JVM (Java Virtual Machine) виконує функцію двигуна для запуску програм Java. Саме JVM насправді викликає основний метод `main()`, присутній у коді Java. JVM є частиною JRE (Java Runtime Environment).

Програми Java запускаються за принципом WORA (Write Once Run Anywhere). Це означає, що програміст може розробляти код Java в одній системі і може очікувати, що він працює в будь-якій іншій системі з підтримкою Java без будь-яких налаштувань. Це все можливо через JVM.

Коли ми компілюємо файл `.java`, компілятором Java генеруються файли з розширенням `.class` які містять байт-код з тими ж назвами класів, які є у файлі `.java`. Цей файл `.class` виконує різні команди коли ми його запускаємо. Опис всіх цих кроків спільно описують весь JVM.

Підсистема навантажувача класів ([Class Loader Subsystem](#))

В основному він відповідає за три види діяльності.

- Завантаження
- Зв'язування
- Ініціалізація

Завантаження([Loading](#)): Завантажувач класу зчитує файл `.class`, генерує відповідні бінарні дані та зберігає їх у області методу. Для кожного `.class` файлу JVM зберігає наступну інформацію в області методу:

- Повноцінне ім'я завантаженого класу та його безпосереднього батьківського класу.
- Чи пов'язаний файл `.class` з класом або інтерфейсом або Enum
- Інформація про модифікатор, змінні та метод тощо.

Після завантаження файлу `.class`, JVM створює об'єкт типу `Class` для представлення цього файлу в купі пам'яті. Зверніть увагу, що цей об'єкт має тип

Class, визначений в пакеті java.lang. Цей об'єкт Class може бути використаний програмістом для отримання інформації про рівень класу, наприклад, імена класів, імена батьківських класів, методів та змінних тощо. Для отримання цього посилання на об'єкт ми можемо використовувати метод getClass () класу Object.

Примітка: Для кожного завантажуваного файлу [.class](#), *лише один об'єкт класу створюється*.

Зв'язування(Linking): виконує перевірку, підготовку та (можливо) дозвіл.

- Перевірка(верифікація, [Verification](#)): вона перевіряє правильність файлу .class, тобто перевіряє, чи правильно цей файл відформатований та чи правильно він згенерований компілятором чи ні. Якщо перевірка не вдається, ми отримуємо виняток під час запуску java.lang.VerifyError.
- Підготовка([Preparation](#)): JVM виділяє пам'ять для змінних класів та ініціалізує пам'ять для значень за замовчуванням.
- Дозвіл ([Resolution](#)): це процес заміни символічних посилань від типу на прямі посилання. Це робиться за допомогою пошуку в області методу, щоб знайти посилання на сутності.

Ініціалізація(Initialization): У цій фазі всі статичні змінні присвоюються їх значенням, визначеним у коді та статичному блоці (якщо такі є). Це виконується зверху вниз у класі та від батька до дитини в ієрархії класів.

Загалом, навантажувачі є трьох класів:

- завантажувач класів завантаження: Кожна реалізація JVM повинна мати завантажувач класів завантажувального класу, здатний завантажувати довірені класи. Він завантажує основні класи API Java, присутні в каталозі JAVA_HOME / jre / lib. Цей шлях в народі відомий як шлях до завантаження. Він реалізований на рідних мовах, таких як C, C ++.
- завантажувач класу розширень: це наслідник завантажувача класу завантаження. Він завантажує класи, наявні в каталогах розширень JAVA_HOME / jre / lib / ext (шлях розширення) або будь-який інший каталог, визначений властивістю системи java.ext.dirs. Він реалізований в java класом sun.misc.Launcher \$ ExtClassLoader.
- завантажувач системного / прикладного класу: це наслідник завантажувача класу розширень. Він несе відповідальність за завантаження класів із шляху класу додатків. Він внутрішньо використовує змінні середовища, яка зберігаються в java.class.path. Він також реалізований на Java класом sun.misc.Launcher \$ AppClassLoader.

Пам'ять JVM

Область методу: в області методу зберігається вся інформація про рівень класу, як найменування класу, ім'я безпосереднього батьківського класу, інформація

про методи та змінні тощо, включаючи статичні змінні. На JVM існує лише одна область методів, і це спільний ресурс.

Купа: Інформація про всі об'єкти зберігається в купі. Є лише одна куча для даного JVM. Також є зона сумісного використання пам'яті(shared resource) на JVM.

Область стеку: для кожного потоку JVM створює один стек під час виконання, який зберігається. Кожен блок цього стека називається записом активації / кадром стека, який зберігає виклики методів. Усі локальні змінні цього методу зберігаються у відповідному кадрі. Після припинення потоку JVM стек буде знищений. Це не спільний ресурс.

Реєстри ПК: зберігають адресу поточної інструкції виконання потоку. Очевидно, що кожен потік має окремі регістри ПК.

Стек Рідних (Native) методів: для кожного потоку створюється окремий нативний стек. У ньому зберігається інформація про метод.

Двигун Виконання (Execution engine)

Двигун Виконання виконує .class (байт-код). Він читає байт-код рядок за рядком, використовує дані та інформацію, наявні в різних областях пам'яті, та виконує інструкції. Його можна класифікувати на три частини:

- Інтерпретатор: Він інтерпретує рядок байт-коду за рядком, а потім виконує. Недоліком тут є те, що коли один метод викликається багаторазово, потрібно щоразу проводити переклад.
- Компілятор Just-in-Time (JIT): Він використовується для підвищення ефективності інтерпретатора. Він збирає весь байт-код і змінює його на власний код, тому щоразу, коли інтерпретатор бачить повторні виклики методу, JIT надає прямий нативний код для цієї частини, тому повторно переводит в машинний код не потрібно, таким чином підвищується ефективність.
- сміттєзбірник: він знищує об'єкти, на які немає посилань.

Java Native Interface (JNI):

Це інтерфейс, який взаємодіє з бібліотеками Native Method і надає рідні бібліотеки (C, C ++), необхідні для виконання. Це дозволяє JVM викликати C / C ++ бібліотеки і викликати бібліотеки C / C ++, які можуть бути специфічними для обладнання.

Бібліотеки рідних методів:

Це колекція "Рідних бібліотек" (C, C ++), які потрібні механізму виконання.

Завантажити JDK або JRE окремо можна, зокрема, на сайтах

<https://java.com/ru/download/>

<https://openjdk.java.net/>

або на офіційному сайті

<https://www.oracle.com/technetwork/java/javase/downloads/index.html>

Установка Java Environment для Windows

1. Завантажити Java8 JDK можна з сайту

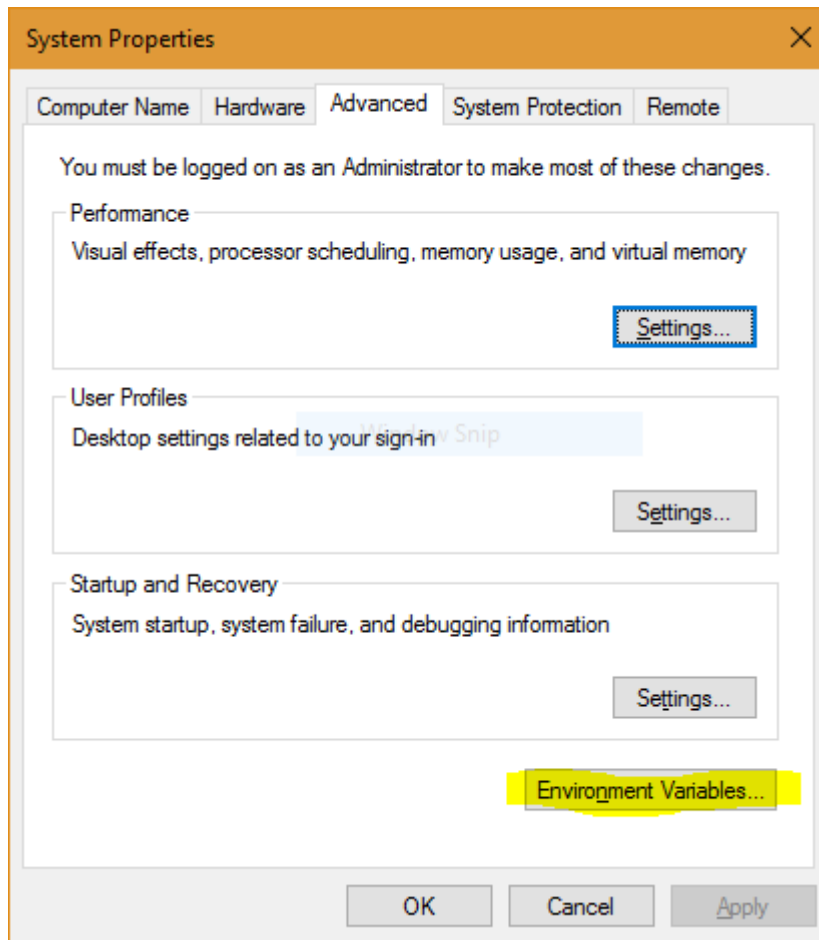
<https://www.oracle.com/technetwork/java/javase/downloads/index.html>

Оберіть версію для потрібної **Windows**

Java SE Development Kit 8u121		
You must accept the Oracle Binary Code License Agreement for Java SE to download this software.		
☐ Accept License Agreement ☒ Decline License Agreement		
Product / File Description	File Size	Download
Linux ARM 32 Hard Float ABI	77.86 MB	jdk-8u121-linux-arm32-vfp-hflt.tar.gz
Linux ARM 64 Hard Float ABI	74.83 MB	jdk-8u121-linux-arm64-vfp-hflt.tar.gz
Linux x86	162.41 MB	jdk-8u121-linux-i586.rpm
Linux x86	177.13 MB	jdk-8u121-linux-i586.tar.gz
Linux x64	159.96 MB	jdk-8u121-linux-x64.rpm
Linux x64	174.76 MB	jdk-8u121-linux-x64.tar.gz
Mac OS X	223.21 MB	jdk-8u121-macosx-x64.dmg
Solaris SPARC 64-bit	139.64 MB	jdk-8u121-solaris-sparcv9.tar.Z
Solaris SPARC 64-bit	99.07 MB	jdk-8u121-solaris-sparcv9.tar.gz
Solaris x64	140.42 MB	jdk-8u121-solaris-x64.tar.Z
Solaris x64	96.9 MB	jdk-8u121-solaris-x64.tar.gz
Windows x86	189.36 MB	jdk-8u121-windows-i586.exe
Windows x64	195.51 MB	jdk-8u121-windows-x64.exe

2. Після завантаження запустить .exe файл і слідуйте інструкціям по встановленню Java на вашу машину. Після встановлення Java на вашу машину, потрібно встановити системні змінні та шляхи.

3. Перейдіть **Control Panel -> System and Security -> System**. Під вкладкою Advanced System Setting знайдіть **Environment Variables**.



4. Тепер потрібно змінити змінну "Path" у якості системних змінних (System variables) так, щоб вона містила посилання на Java середвище. Для цього оберить змінну "Path" та клікните на кнопку Edit як на малюнку.
5. Ви побачите список різних шляхів, клікните на кнопку New та додайте шлях до директрії де встановлена java. За замовченням, java встановлюється в "C:\Program Files\Java\jdk\bin" або "C:\Program Files(x86)\Java\jdk\bin". В інших випадках додайте ваш шлях.
6. Клікнувши OK, збережить конфігурацію. Для перевірки, що все встановлено як треба, наберить в терміналі `javac -version`. Ви побачите, яку версію java ви встановили.

7. Для перевірки того, що встановлений компілятор наберіть *javac* в командному рядку. Ви побачите рядок з інформацією про компілятор.

Установка Java Environment для Linux

Для ОС linux існує декілька варіантів установки java. Найбільш природній для цієї ОС та кращий варіант – через apt install в терміналі. Для того щоб зайвий раз не реєструватись на сайті oracle встановимо OpenJDK. OpenJDK – це відкрита та open source версія мови Java.

1. Зайдіть в термінал (**Application -> Accessories -> Terminal**).
2. Наберіть команду

```
sudo apt-get install openjdk-8-jdk
```
3. Для встановлення змінної оточення “JAVA_HOME” (Environment Variable) наберіть в терміналі наступну команду (вказавши або цей шлях по замовченню або власний)

```
export JAVA_HOME = /usr/lib/jvm/java-8-openjdk
```
4. Для встановлення змінної оточення “PATH” (Environment Value) наберіть в терміналі наступну команду (вказавши або цей шлях по замовченню або власний)

```
export PATH = $PATH:/usr/lib/jvm/java-8-openjdk/bin
```
5. Для перевірки установки наберіть *java -version* в терміналі.

Ці шляхи можна оминати якщо відразу встановити IDE для мови, вказавши, що разом потрібно встановити і відповідне середовище.

Серед найбільш популярних IDE для Java можна виділити наступні:

Eclipse

IntelliJ IDEA

Netbeans

ВИКОНАННЯ JAVA-ПРОГРАМИ

Програма, написана на мові високого рівня, до якого належить і мова Java, так званий вихідний модуль, не може бути відразу ж виконана.

Її спочатку потрібно відкомпілювати, тобто перевести в послідовність машинних команд - об'єктний модуль. Але і він, як правило, не може бути відразу ж виконаний: об'єктний модуль потрібно ще скомпонувати з бібліотеками використовуваними в модулі функцій і дозволити перехресні посилання між секціями об'єктного модуля, отримавши в результаті завантажуваний модуль - повністю готову до виконання програму.

Вихідний модуль, написаний на Java, не може уникнути цих процедур, але тут проявляється головна особливість технології Java - програма компілюється відразу в машинні команди, але не команди якогось конкретного процесора, а в команди так званої віртуальної машини Java (JVM, Java Virtual Machine). Віртуальна машина Java - це сукупність команд разом з системою їх виконання. Команди JVM короткі, більшість з них має довжину 1 байт, чому команди JVM називають байт-кодами (bytecodes), хоча є команди довжиною 2 і 3 байти.

Інша особливість Java - всі стандартні функції, викли-ваемі в програмі, підключаються до неї тільки на етапі виконання, а не включаються в байт-коди. Як кажуть фахівці, від-ходить динамічна компоновка (dynamic binding). Це теж сильно зменшує обсяг скомпільованій.

Отже, на першому етапі програма, написана на мові Java, перекладається компілятором в байт-коди. Ця компіляція не залежить від типу будь-якого конкретного процесора і архітектури якогось конкретного комп'ютера. Вона може бути виконана один раз відразу ж після написання програми. Байт-коди записуються в одному або декількох файлах, можуть зберігатися в зовнішній пам'яті або передаватися по мережі. Це особливо зручно завдяки невеликому розміру файлів з байт-кодами. Потім отримані в результаті компіляції байт-коди можна виконувати на будь-якому комп'ютері, що має систему, що реалізовує JVM. При цьому не має значення ні тип процесора, ні архітектура комп'ютера.

Інтерпретація байт-кодів і динамічна компоновка значно уповільнюють виконання програм. Це не має значення в тих ситуаціях, коли байт-коди передаються по мережі, мережа все одно повільніше будь-якій інтерпретації, але в інших ситуаціях потрібно потужний і швидкий комп'ютер. Тому постійно йде вдосконалення інтерпретаторів в бік збільшення швидкості інтерпретації. Розроблено JIT-компілятори (Just-In-Time), що запам'ятовують вже інтерпретовані ділянки коду в машинних командах процесора і просто виконують ці ділянки при повторному зверненні, наприклад, в циклах. Це значно збільшує швидкість повторюваних обчислень. При виконанні програм

Java потрібна інтерпретація команд JVM в команди конкретного процесора, а значить, потрібна програма-інтерпретатор, причому для кожного типу процесорів і для кожної архітектури комп'ютера потрібен свій інтерпретатор.

Це завдання вже вирішена практично для всіх комп'ютерних платформ. Крім реалізації JVM для виконання байт-кодів на комп'ютері необхідний набір функцій, що викликаються з байт-кодів і динамічно компонується з байт-кодами. Цей набір оформлюється у вигляді бібліотеки класів Java, що складається з одного або декількох пакетів. Кожна функція може бути записана байт-кодами, але, оскільки вона буде зберігатися на конкретному комп'ютері, її можна записати прямо в системі команд цього комп'ютера, уникнувши тим самим інтерпретації байт-кодів. Такі функції на-викликають "рідними" методами (native methods). Застосування "рідних" методів прискорює виконання програми.

Спочатку SUN Microsystems - творець технології Java, а потім Oracle (теперішній її ліцензіар) - безкоштовно розповсюджує набір необхідних програмних інструментів для повного циклу роботи з цією мовою програмування: компіляції, інтерпретації, налагодження, що включає і багату бібліотеку класів, під назвою JDK (Java Development Kit).

Набір програм і класів JDK містить:

- компілятор `javac` з вихідного тексту в байт-коди;
- інтерпретатор `java`, що містить реалізацію JVM;
- програму перегляду аплетів `appletviewer`, яка замінює браузер;
- відладчик `jdb`;
- дизасемблер `javap`;
- програму архівації та стиснення `jar`;
- програму збору документації `javadoc`;
- програму `javaH` генерації заголовних файлів мови C;
- програму `javaKey` додавання електронного підпису;
- програму `native2ascii`, перетворюючу бінарні файли в текстові;
- програми `rmic` і `rmiregistry` для роботи з віддаленими об'єктами;
- програму `serialver`, визначає номер версії класу;
- бібліотеки та заголовки "рідних" методів;
- бібліотеку класів Java API (Application Programming Interface).

Написати програму на Java можна в будь-якому текстовому редакторі, зберігши файл в текстовому форматі з розширенням `java`.

Нехай, наприклад, іменем файлу буде `MyProgram.java`, а сам файл збережений в поточному каталозі.

Після створення цього файлу з командного рядка викликається компілятор `javac` і йому передається вихідний файл як параметр:

```
javac MyProgram.java
```

Компілятор створює в тому ж каталозі по одному файлу на кожен клас, описаний в програмі, називаючи кожен файл име-ньому класу з розширенням class. Припустимо, є тільки один клас, названий MyProgram, тоді отримуємо файл з ім'ям MyProgram.class, що містить байт-коди. Якщо компіляція пройшла успішно, то компілятор нічого не повідомить, на екрані з'явиться тільки запрошення ОС. Якщо ж компілятор помітить помилки, то він виведе на екран повідомлення про них. Велика перевага компілятора JDK в тому що він "відловлює" багато помилок і видає докладні і зрозумілі повідомлення про них. Далі з командного рядка викликається інтерпретатор байт-кодів java, якому передається файл з байт-кодами, причому його ім'я записується без розширення:

```
java MyProgram
```

На екрані з'являється висновок результатів роботи програми або повідомлення про помилки часу виконання.

Програма "Hello, world" на Java

```
class HelloWorld
{
    public static void main(String[] args)
    {
        System.out.println
            ("Hello, World!");
    }
}
```

Можна помітити ряд істотних особливостей мови Java:

1. Будь-яка програма складається з одного або декількох класів.
2. Початок класу відзначається службовим словом class, за яким слідує ім'я класу, вибираємо довільно, в даному випадку - HelloWorld. Все, що міститься в класі, записується в фігур-них дужках і становить тіло класу (class body).
3. Всі дії проводяться за допомогою методів обробки інформації, які звуться просто метод (method), які є аналогами "функцій", що застосовуються в інших мовах. З точки зору C++ - це методи класу HelloWorld.
4. Методи розрізняються за своїми назвами. Один з методів обов'язково повинен називатися main, з нього починається виконання програми. У даній програмі тільки один метод - main.
5. Як і належить функції, метод завжди видає в результаті (частіше говорять, повертає (returns)) тільки одне значення, тип якого обов'язково вказується перед ім'ям методу. Метод може і не повертати ніякого значення, граючи роль процедури, як в даному випадку. Тоді замість типу значення, що повертається записується слово void, як це і зроблено в прикладі.
6. Після імені методу в дужках, через кому, перераховуються аргументи

(arguments), або параметри методу. Для кожного аргументу вказуються його тип і через пробіл ім'я. У прикладі тільки один аргумент, його тип - масив, що складається з рядків символів. Рядок символів - це вбудований в Java API тип string, а квадратні дужки - ознака масиву. Ім'я масиву може бути довільним, в прикладі вибрано ім'я args.

7. Перед типом повертається методом значення можуть бути записані модифікатори (modifiers). У прикладі їх два: слово public означає, що цей метод доступний звідусіль; слово static забезпечує, що незважаючи на те що клас не завантажений все одно можна викликати метод main () на самому початку виконання програми. Модифікатори взагалі необов'язкові, але для методу main () вони необхідні.

Єдина дія, яке виконує метод main () в прикладі, полягає у виклику іншого методу зі складним ім'ям System.out.println і передачі йому на обробку одного аргументу - текстової константи "Hello, World!". Текстові константи записуються в лапках, які є тільки обмежувачами і не входять до складу тексту.

Складене ім'я System.out.println означає, що в класі System, що входить в Java API, визначається змінна з ім'ям out, що містить екземпляри одного з класів Java API, класу PrintStream в якому є метод println ().

Дія методу println () полягає у виведенні свого аргументу у вихідний потік, пов'язаний зазвичай з виведенням на екран текстів терміналу. Після виведення курсор переходить на початок наступного рядку екрану, на що вказує закінчення ln, слово println - скорочення слів print line. До складу Java API входить також метод print (), який залишає курсор в кінці виведеного рядку.

Мова Java розрізняє малі та великі літери (case-sensitive), тому імена main, Main, MAIN різні з "точки зору" компілятора Java. У прикладі необхідно писати String, System з великої літери, а main з малої.

Але всередині текстової константи неважливо, писати World або world, компілятор взагалі не "дивиться" на неї, різниця буде видна тільки на екрані.

Існують певні стилістичні угоди про імена:

- імена класів починаються з великої літери; якщо ім'я має декілька слів, то кожне слово починається з прописної букви;
- імена методів і змінних починаються з малої літери; якщо ім'я містить кілька слів, то кожне наступне слово починається з малої літери;
- імена констант записуються повністю прописними літерами; якщо ім'я складається з декількох слів, то між ними ставиться знак підкреслення.

Звичайно, ці правила не є обов'язковими, але сильно полегшують розуміння коду і надають програмі характерний для Java стиль.

Отже, програма написана в будь-якому текстовому редакторі, наприклад Notepad. Тепер її потрібно зберегти в файлі, ім'я якого збігається з ім'ям класу, що містить метод main (), і дати імені файлу розширення java, що

дозволить системі виконання Java швидко знаходити метод `main ()` для початку роботи, просто відшукавши клас, що збігається з ім'ям файлу.

В даному випадку збережемо програму в файлі з іменем *HelloWorld.java* в поточній директорії. Потім потрібно зробити виклик компілятора, передаючи йому ім'я файлу в якості аргументу:

```
javac HelloWorld.java
```

Компілятор створить файл з іменем *HelloWorld.class* в поточному каталозі.

Осталось викликати інтерпретатор, передаючи йому в якості аргументу ім'я класу (а не файлу):

```
java HelloWorld
```

На екрані повинен з'явитись напис:

```
Hello, World!
```

Коментарі

У програму можна вставити коментарі, тобто текст, що ігнорується компілятором. Вони дуже корисні для пояснень по ходу програми. Також під час налагодження можна виключати з дій один або кілька операторів, позначивши їх символами коментаря, як го-ворят програмісти, "закоментувати" їх. Коментарі вводяться таким чином:

- за двома похилими рисами поспіль `//` (без пробілу між ними) починається коментар, що триває до кінця рядка;

- за похилою рисою і зірочкою `/*` починається коментар, який може займати кілька рядків, до зірочки і похилій риски `*/` (без пробілів між цими знаками).

Коментарі дуже зручні для читання і розуміння коду, вони перетворюють програму в документ, що описує її дії. Програму з хорошими коментарями називають самодокументованою. Тому в Java введені коментарі третього типу, а до складу JDK - програма `javadoc`, яка витягує ці коментарі в окремі файли формату HTML і створює гіперпосилання між ними: за похилою рисою і двома зірочками поспіль `/**` (без пробілів) починається коментар, який може займати декілька рядків до зірочки (однієї) і похилій риски `*/` і обробляються програмою `javadoc`. У такий коментар можна вставити вказівки програмі `javadoc`, які починаються з символу `@`.

Саме так створюється документація до JDK.

```
/**
```

```
 * Програма друку Хелло Ворлд
```

```
 * @author Name Surname (or Nick)
```

```
 * @version 1.0 (version of program)
```

```
 */
```

```
class HelloWorld2
```

```
{ // HelloWorld – ім'я класу
```

```
    // Головний метод класу HelloWorld2 – стартова точка
```

```

    public static void main(String[] args)
    { // args не використовується
/* Цей метод виводить свій аргумент на екран */
        System.out.println("Hello, World !");
/* Наступний метод не буде виконуватись
        System.out.println("Farewell, 20th Century!");
*/

    }
}

```

Константи

В Java існують константи різних типів в різних видах.

Цілі

Цілі константи можна записувати в трьох системах числення:

- в десятковій формі: +5, -7, 12345678;
- в вісімковій формі, починаючи з нуля: 027, -0326, 0777; в запису таких констант неприпустимі цифри 8 і 9;
- в шістнадцятковій формі, починаючи з нуля і латинської літери x або X: 0xff0a, 0xFC2D, 0x45a8, 0X77FF (тут малі та великі літери не розрізняються);

Цілі константи зберігаються в форматі типу int. В кінці цілої константи можна записати букву прописну L або малу l, тоді константа буде зберігатися в довгому форматі типу long: + 25L, -0371, 0xffL, 0XDFDF1.

Не використовуйте при запису довгих цілих констант малу латинську букву l, її легко сплутати з одиницею.

Дійсні

Дійсні константи записуються тільки в десятковій системі числення в двох формах:

- с фіксованою точкою: 37.25, -128.678967, +27.035;
- з плаваючою точкою: 2.5e34, -0.345e-25, 37.2E+4 (допускається писати рядкову або прописну латинську букву E, пробіли і дужки всередині числа неприпустимі);

В кінці дійсної константи можна поставити букву F або f, тоді константа буде зберігатися в форматі типу float: 3.5f, -45.67F, 4.7e-5f. Можна приписати і букву D (або d): 0.045D, -456.77889d, яка б означала тип double, але це зайве, по-скільки дійсні константи і так зберігаються в форматі типу double.

Символи

Для запису символів використовуються такі форми.

1. Друковані символи можна записати в апострофа: 'a', 'N', '?'.

2. Керуючі символи записуються в апострофах зі зворотним похилою рисою:

- '\ n' - символ перекладу рядка newline з кодом ASCII 10;
- '\ r' - символ повернення каретки CR з кодом 13;
- '\ f' - символ перекладу сторінки FF з кодом 12;
- '\ b' - символ повернення на крок BS з кодом 8;
- '\ t' - символ горизонтальної табуляції HT з кодом 9;
- '\\ ' - зворотна похила риска;
- '\ "' - лапки;
- '\ ' - апостроф.

3. Код будь-якого символу з десятковою маються на кодуванням від 0 до 255 можна задати, записавши його не більше ніж трьома цифрами в восьмеричній системі числення в апострофі після зворотної похилої риси: '\123' - буква S, '\346' - буква Ж в кодуванні CP1251. Не рекомендується використовувати цю форму запису для друкованих та керуючих символів, перерахованих в попередньому пункті, по-скільки компілятор переведе восьмеричну запис в зазначену вище форму.

4. Код будь-якого символу в кодуванні Unicode набирається в апострофах після зворотної похилої риси і латинської літери u чотирма шестнадцатерічними цифрами: '\ u0053' - буква S, '\ u0416' - буква Ж.

Символи зберігаються в форматі типу char.

Примітка: Прописні російські букви в кодуванні Unicode займають діапазон від '\ u0410' - велика літера А, до '\ u042F' - велика літера Я, малі літери - від '\ u0430' - буква а, до '\ 044F' - буква я . В якій би формі не записувались символи, компілятор переводить їх в Unicode, включаючи і початковий текст програми.

Рядки

Рядки символів полягають в лапки. Керуючі сим-воли та коди записуються в рядках точно так же, зі зворотним на-клонів рисою, але, зрозуміло, без апострофів, і надають ту саму дію. Рядки можуть розташовуватися тільки на одному рядку ис-перехідного коду, не можна відкриває лапки поставити на одному рядку, а закриває - наступного.

Ось деякі приклади:

"Це рядок \ n з перенесенням"

"\" Динамо \"- Чемпіон!"

Рядки символів не можна починати на одному рядку вихідного коду, а закінчувати на інший.

Для строкових констант визначена операція зчеплень, обо-значаючих плюсом.

"Зчеплення" + "рядків" дає в результаті рядок "Зчеплення рядків".

Щоб записати довгий рядок у вигляді однієї строкової кон-константи, потрібно після закриває лапки на першій і наступних рядках поставити знак плюс, а компілятор збере рядки в одну строкову константу, наприклад:

"Одна строкова константа, записана" +
"На двох рядках вихідного тексту"

Той, хто спробує виводити символи в кодуванні Unicode, повинен пам'ятати, що Windows 95/98 / ME взагалі не працює з Unicode, а Windows NT / 2000 використовує для виведення у вікно Command Prompt шрифт Terminal, в якому російські букви, розташовані в початкових кодах Unicode, в кодуванні cp866, і розкидані по інших сегментах Unicode.

Не всі шрифти Unicode містять накреслення (glyphs) всіх символів, тому будьте обережні при виведенні рядків в кодуванні Unicode. Тому використовуйте Unicode безпосередньо тільки в крайніх випадках.

Імена

Імена (names) змінних, класів, методів та інших об'єктивним тов можуть бути простими (загальна назва - ідентифікатори (idenifiers)) і складовими (qualified names). Ідентифікатори в Java складаються з так званих букв Java (Java letters) і арабських цифр 0 - 9, причому першим символом ідентифікатора не може бути цифра. (Дійсно, як зрозуміти запис 2e3: як число 2000,0 або як ім'я змінної?) В число букв обов'язково входять великі та малі латинські букви, знак долара \$ і знак підкреслення _, а так само символи національних алфавітів.

Не вказуйте в іменах знак долара. Компілятор Java використовує його для запису імен вкладених класів.

Ось приклади правильних ідентифікаторів:

```
a1    my_var    var3_5    _var    veryLongVarName  
aName theName  a2Vh36kBnMt456dX
```

В іменах краще не використовувати малу літеру l, яку легко сплутати з одиницею, і букву o, яку легко прийняти за нуль.

Службові слова Java, такі як class, void, static, зарезервовані, їх не можна використовувати в якості ідентифікаторів своїх об'єктів.

Складене ім'я (qualified name) - це кілька ідентифікаторів, між якими ставлять крапку без пробілів, наприклад, ім'я System.out.println, яке вже зустрічалося.

Приклад.

```
/*  
    Math Hello world  
*/  
class DoMath  
{  
    public static void main(String[] args)  
    {  
        int m;
```

```

        m=101;
        int n = 2*m+1;
        System.out.print(«виводимо значення змінної»);
        System.out.println(«n=»,n);

    }
}

```

Байт-код

Якщо скомпілювати код

```

public class h1{
    public static void main(String[] args){
        int m = 2;
        int n = m * m * 3 + 1;
        System.out.println("Hi"+String.valueOf(m));
    }
}

```

}

За допомогою команди

javap -c h1

То можна отримати байт-код даної програми:

Compiled from "h1.java"

```

public class h1 {

```

```

    public h1();

```

Code:

```

        0: aload_0

```

```

        1: invokespecial #1

```

```

// Method

```

```

java/lang/Object."<init>":()V

```

```

        4: return

```

```

public static void main(java.lang.String[]);

```

Code:

```

        0: iconst_2

```

```

        1: istore_1

```

```

        2: iload_1

```

```

        3: iload_1

```

```

        4: imul

```

```

        5: iconst_3

```

```

        6: imul

```

```

        7: iconst_1

```

```

        8: iadd

```

```

        9: istore_2

```

```

        10: getstatic      #2                // Field
java/lang/System.out:Ljava/io/PrintStream;
        13: iload_1
        14: invokestatic  #3                // Method
java/lang/String.valueOf:(Ljava/lang/String;
        17: invokedynamic #4,  0                // InvokeDynamic
#0:makeConcatWithConstants:(Ljava/lang/String;)Ljava/lang/String;
        22: invokevirtual #5                // Method
java/io/PrintStream.println:(Ljava/lang/String;)V
        25: return
    }

```

Виведення

Найбільш простий спосіб взаємодії з користувачем представляє консоль: ми можемо виводити на консоль деяку інформацію або, навпаки, зчитувати з консолі деякі дані. Для взаємодії з консоллю в Java застосовується клас System, а його функціональність власне забезпечує консольне введення і виведення.

Висновок на консоль

Для створення потоку виведення в клас System визначено об'єкт out. В цьому об'єкті визначено метод println, який дозволяє вивести на консоль деяке значення з подальшим переведенням консолі на наступний рядок. наприклад:

```

public class Program {

    public static void main (String [] args) {

        System.out.println ( "Hello world!");
        System.out.println ( "Bye world ...");
    }
}

```

У метод println передається будь-яке значення, як правило, рядок, яке треба вивести на консоль. І в даному випадку ми отримаємо наступний висновок:
Hello world!

Bye world ...

При необхідності можна і не переводити курсор на наступний рядок. В цьому випадку можна використовувати метод System.out.print (), який аналогічний println за тим винятком, що не чинить перекладу на наступний рядок.

```

public class Program {

    public static void main (String [] args) {

```

```
System.out.print ( "Hello world!");  
System.out.print ( "Bye world ...");  
}  
}
```

Консольне виведення даної програми:

Hello world! Bye world ...

Але за допомогою методу `System.out.print` також можна здійснити переведення каретки на наступний рядок. Для цього треба використовувати escape-послідовність `\n`:

```
System.out.print ( "Hello world \n");
```

Нерідко необхідно підставляти в рядок якісь дані. Наприклад, у нас є два числа, і ми хочемо вивести їх значення на екран. У цьому випадку ми можемо, наприклад, написати так:

```
public class Program {
```

```
    public static void main (String [] args) {
```

```
        int x = 5;
```

```
        int y = 6;
```

```
        System.out.println ( "x =" + x + "; y =" + y);
```

```
    }
```

```
}
```

Результат програми:

x = 5; y = 6

Але в Java є також функція для форматованого виведення, успадкована від мови C: `System.out.printf ()`. З її допомогою ми можемо переписати попередній приклад наступним чином:

```
int x = 5;
```

```
int y = 6;
```

```
System.out.printf ( "x=%d; y=%d \n", x, y);
```

В даному випадку символи `%d` позначають специфікатор, замість якого підставляє один з аргументів. Специфікаторів і відповідних їм аргументів може бути безліч. В даному випадку у нас тільки два аргументи, тому замість першого `%d` підставляє значення змінної `x`, а замість другого - значення змінної `y`. Сама буква `d` означає, що даний специфікатор буде використовуватися для виведення цілочисельних значень.

Крім специфікатор `%d` ми можемо використовувати ще ряд специфікаторів для інших типів даних:

`%X`: для виведення шістнадцяткових чисел

% F: для виведення чисел з плаваючою точкою
% E: для виведення чисел у експоненційній формі, наприклад, 1.3e + 01
% C: для виведення одиночного символу
% S: для виведення строкових значень
наприклад:

```
public class Program {  
  
    public static void main (String [] args) {  
  
        String name = "Tom";  
        int age = 30;  
        float height = 1.7f;  
  
        System.out.printf ( "Name:% s Age:% d Height:% .2f \ n", name, age, height);  
    }  
}
```

При виведенні чисел з плаваючою точкою ми можемо вказати кількість знаків після коми, для цього використовуємо специфікатор на% .2f, де .2 вказує, що після коми буде два знака. У підсумку ми отримаємо наступний висновок:
Name: Tom Age: 30 Height: 1,70

Введення з консолі

Для отримання введення з консолі в класі System визначено об'єкт in. Однак безпосередньо через об'єкт System.in не дуже зручно працювати, тому, як правило, використовують клас Scanner, який, в свою чергу використовує System.in. Наприклад, напишемо маленьку програму, яка здійснює введення чисел:

```
import java.util.Scanner;  
  
public class Program {  
  
    public static void main (String [] args) {  
  
        Scanner in = new Scanner (System.in);  
        System.out.print ( "Input a number:");  
        int num = in.nextInt ();  
  
        System.out.printf ( "Your number:% d \ n", num);  
        in.close ();  
    }  
}
```

```
}  
}
```

Так як клас Scanner знаходиться в пакеті java.util, то ми спочатку його імпортуємо з допомогою інструкції import java.util.Scanner.

Для створення самого об'єкта Scanner в його конструктор передається об'єкт System.in. Після цього ми можемо отримувати вводяться значення. Наприклад, в даному випадку спочатку виводимо запрошення до вводу і потім отримуємо вводиться число в змінну num.

Щоб отримати введене число, використовується метод in.nextInt () ;, який повертає введене з клавіатури цілочисельне значення.

Приклад роботи програми:

Input a number: 5

Your number: 5

Клас Scanner має ще ряд методів, які дозволяють отримати введені користувачем значення:

next (): зчитує введений рядок до першого пробілу

nextLine (): зчитує всю введену рядок

nextInt (): зчитує введене число і

nextDouble (): зчитує введене число double

nextBoolean (): зчитує значення boolean

nextByte (): зчитує введене число byte

nextFloat (): зчитує введене число float

nextShort (): зчитує введене число short

Тобто для введення значень кожного примітивного типу в класі Scanner визначений свій метод.

Наприклад, створимо програму для введення інформації про людину:

```
import java.util.Scanner;
```

```
public class Program {
```

```
public static void main (String [] args) {
```

```
Scanner in = new Scanner (System.in);
```

```
System.out.print ( "Input name:");
```

```
String name = in.nextLine ();
```

```
System.out.print ( "Input age:");
```

```
int age = in.nextInt ();
```

```
System.out.print ( "Input height:");
```

```
float height = in.nextFloat ();
```

```
System.out.printf ( "Name:% s Age:% d Height:% .2f \ n", name, age, height);
```

```
in.close ();
```

```
}  
}
```

Тут послідовно вводяться дані типів String, int, float і потім всі введені дані разом виводяться на консоль. Приклад роботи програми:

Input name: Tom

Input age: 34

Input height: 1,7

Name: Tom Age: 34 Height: 1,70

Зверніть увагу для введення значення типу float (те ж саме відноситься до типу double) застосовується число "1,7", де роздільником є кома, а не "1.7", де роздільником є крапка. В даному випадку все залежить від поточної мовної локалізації системи. У моєму випадку українська локалізація, відповідно вводити необхідно числа, де роздільником є кома. Те ж саме стосується багатьох інших локалізацій, наприклад, німецької, французької і т.д., де застосовується кома.

Налагодження програми Java на Java Eclipse

Найшвидший спосіб налагодження програми Java - це використання додатку Package Explorer. У вкладці Package Explorer

- Клацніть правою кнопкою миші клас java, який містить основний метод.
- Виберіть Debug as → Java Program.

Таку ж дію можна виконати за допомогою програми Package Explorer, вибравши клас, який містить основний метод, і натиснути Alt + Shift + D, J.

Будь-які дії, згадані вище, створюють нову конфігурацію налагодження і використовують її для запуску програми Java.

Якщо конфігурація налагодження вже створена, ви можете використовувати її для запуску програми Java, вибравши в меню «Виконати» конфігурації налагодження, натиснувши назву конфігурації налагодження, а потім натиснувши кнопку налагодження.

Пункт меню «Налагодження» у меню «Виконати» може використовуватися для перезапуску програми java, яка раніше була запущена в режимі налагодження.

Клавіша швидкого доступу для запуску раніше запущеної програми Java в режимі налагодження - F11. Коли програма java запускається в режимі налагодження, користувачам пропонується перейти на перспективу налагодження. Перспектива налагодження пропонує додаткові представлення даних, які можна використовувати для усунення неполадок програми.

Редактор java дозволяє користувачам розміщувати точки перерви в коді Java. Щоб встановити точку розриву, в області редактора клацніть правою кнопкою миші на панелі маркера та виберіть пункт «Переключити точку розриву».

Точки розриву відображаються на маркері. Вони також помітні в Перегляді

точок зламу.

Коли виконання коду досягне точки розриву, JVM призупиняє виконання програми. У цей момент ви можете перевірити пам'ять та виконати програму контрольованим чином.

Коли виконання програми призупинено, вигляд налагодження може використовуватися для перевірки стеку викликів.

Перегляд змінних може використовуватися для перевірки значення змінних.

У меню запуску є пункти меню, які дозволяють відновити виконання, перейти через рядок коду, перейти до функції тощо.